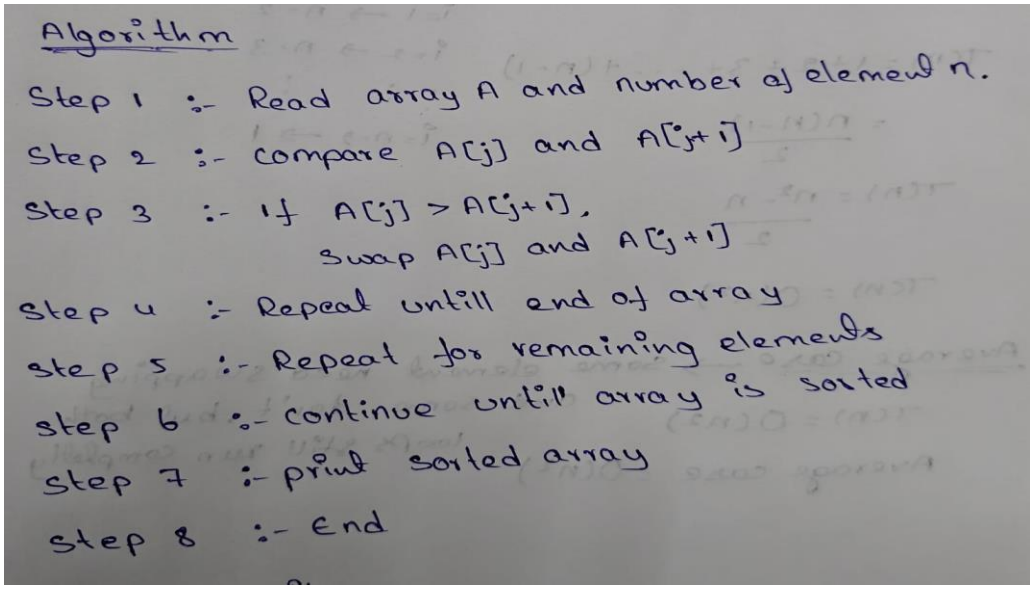
Practical - 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1)Bubble Sort

Algorithm:



Program:

```
#include <iostream>
  using namespace std;
  class BubbleSort
  {
  public:
     int arr[100], n;
     void arrayInput()
     {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
           cin >> arr[i];
        }
     }
     void printArray(){
     for (int i = 0; i < n; i++){
           cout << arr[i] << " ";
        }
```

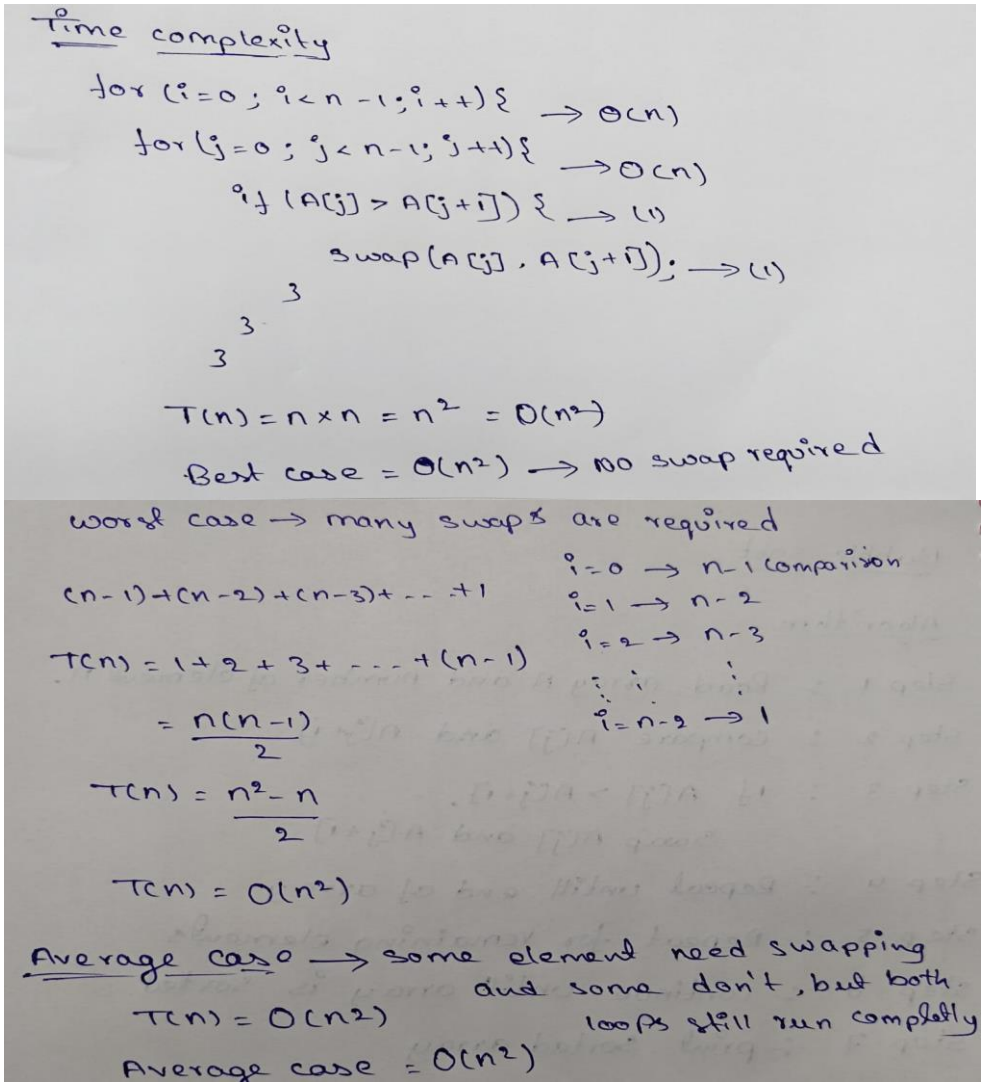
```
        cout << endl;
    }
  void logic(){
     int temp;
        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - i - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};
int main()
{
  BubbleSort bs;
  bs.arrayInput();
    cout << "\nBefore Sorting: ";
  bs.printArray();
  bs.logic();
    cout << "After Sorting: ";
  bs.printArray();
    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\bubble sort.exe"
Enter number of elements: 6
Enter the elements: 12
9
8
22
2
7

Before Sorting: 12 9 8 22 2 7
After Sorting: 2 7 8 9 12 22
```

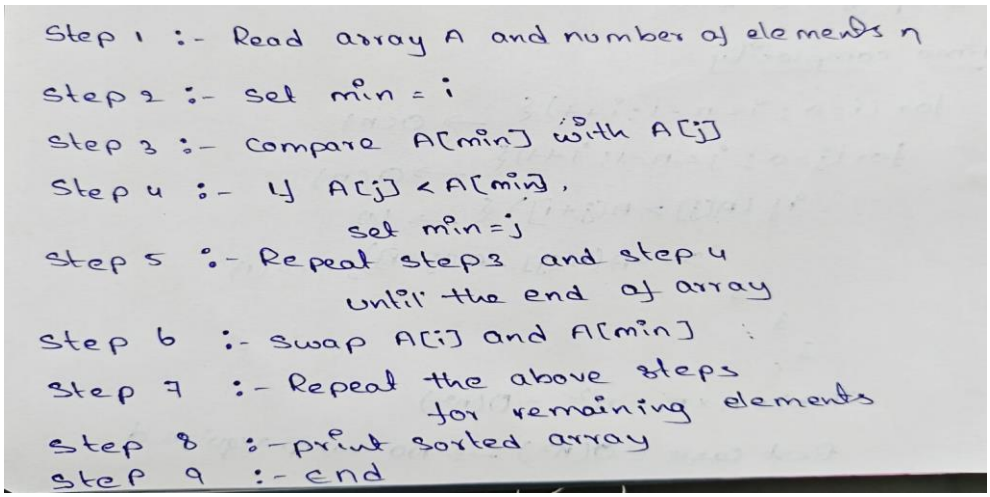
Name: Chinni Thabitha
Enrollment Number:92460118038

Time Complexity:



2) Selection sort:

Algorithm:



Name: Chinni Thabitha
Enrollment Number:92460118038

Program:

```
#include <iostream>
using namespace std;
class SelectionSort{
public:
    int arr[100], n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void logic(){
        int i, j, min, temp;
        for (i = 0; i < n - 1; i++){
            min = i;
            for (j = i + 1; j < n; j++){
                if (arr[j] < arr[min]){
                    min = j;
                }
            }
Swap(arr[i],arr[min]);
        }
    }
    void printArray() {
        for (int i = 0; i < n; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
```

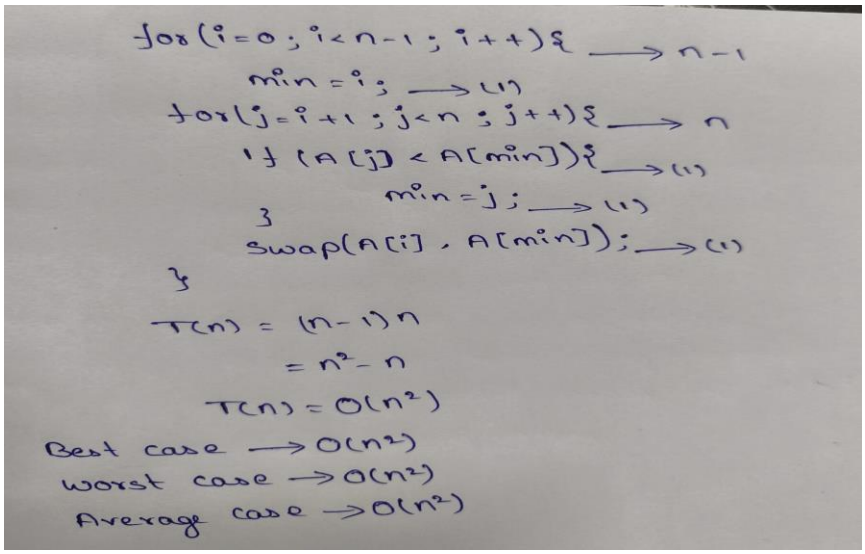
Name: Chinni Thabitha
Enrollment Number:92460118038

```
};
int main()
{
    SelectionSort ss;
    ss.arrayInput();
    cout << "Before Sorting: ";
    ss.printArray();
    ss.logic();
    cout << "After Sorting: ";
    ss.printArray();
    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\selectionsort.exe"
Enter number of elements: 6
Enter the elements: 12
9
2
8
31
1
Before Sorting: 12 9 2 8 31 1
After Sorting: 1 2 8 9 12 31
```

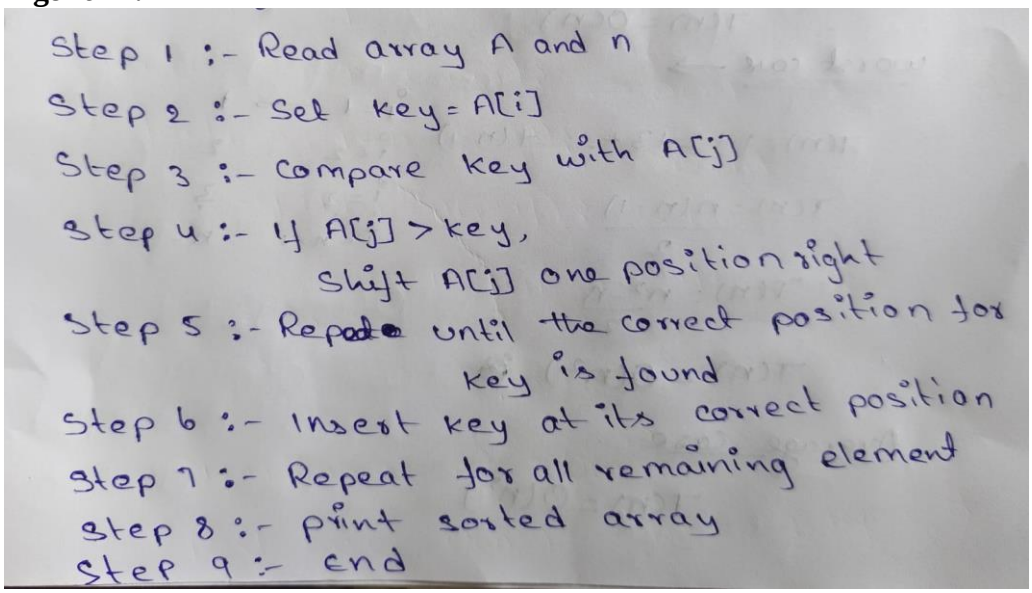
Time complexity:



Name: Chinni Thabitha
Enrollment Number:92460118038

3) Insertion sort:

Algorithm:



Program:

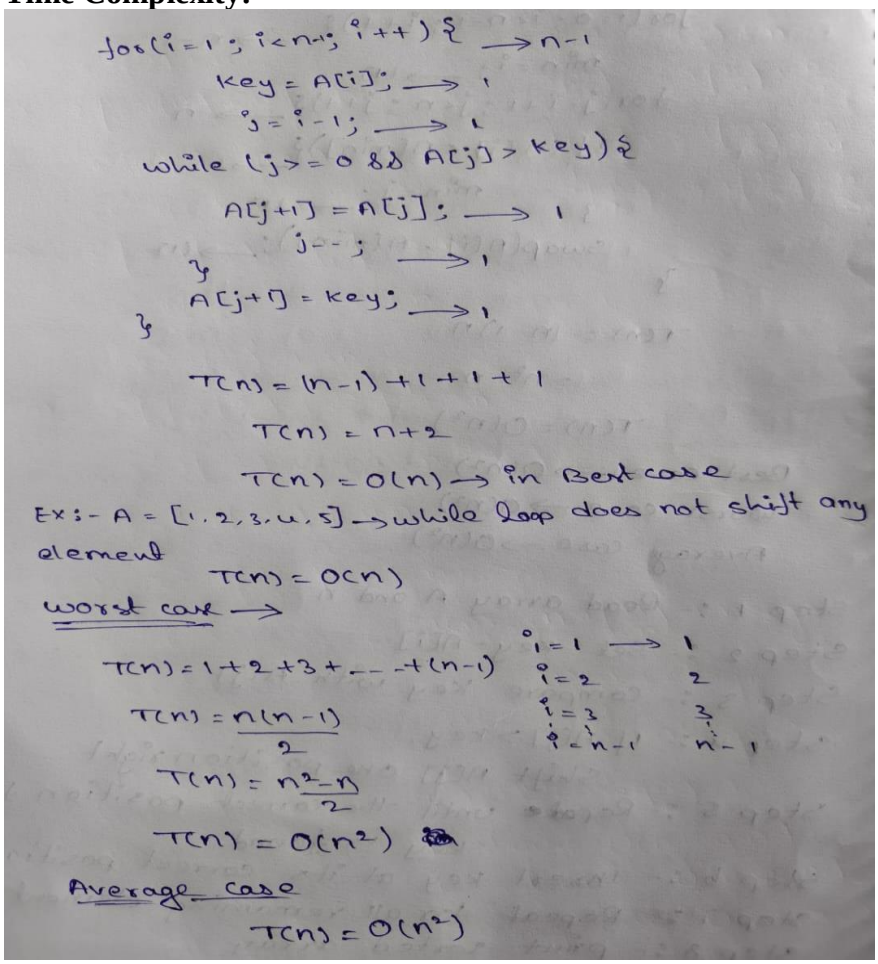
```
#include <iostream>
using namespace std;
class InsertionSort
{
public:
   int arr[100], n;
   void arrayInput()
   {
      cout << "Enter number of elements: ";
      cin >> n;

      cout << "Enter the elements: ";
      for (int i = 0; i < n; i++)
      {
         cin >> arr[i];
      }
   }
   void logic()
   {
      int i, j, key;

      for (i = 1; i < n; i++)
      {
         key = arr[i];
         j = i - 1;

         while (j >= 0 && arr[j] > key)
         {
```

```
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    InsertionSort is;

    is.arrayInput();

    cout << "Before Sorting: ";
    is.printArray();

    is.logic();

    cout << "After Sorting: ";
    is.printArray();

    return 0;
}
```

Output:

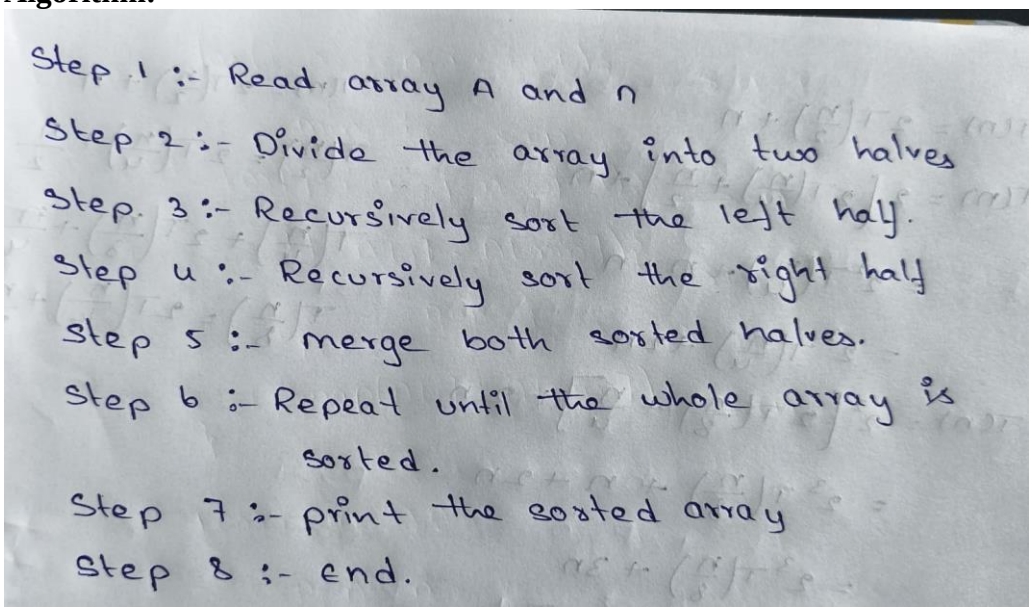
```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\insertionsort.exe"
Enter number of elements: 6
Enter the elements: 12
9
3
32
4
15
Before Sorting: 12 9 3 32 4 15
After Sorting: 3 4 9 12 15 32
```

Time Complexity:



4)Merge Sort:

Algorithm:



Program:

```
#include <iostream>
using namespace std;
class MergeSort{
public:
   int arr[100],n;
   void arrayInput(){
      cout << "Enter number of elements: ";
      cin >> n;
      cout << "Enter the elements: ";
      for (int i = 0; i < n; i++){
         cin >> arr[i];
      }
   }
   void merge(int low, int mid, int high){
      int temp[100];
      int i = low;
      int j = mid + 1;
      int k = low;
      while (i <= mid && j <= high){
         if (arr[i] < arr[j]){
            temp[k] = arr[i];
            i++;
         }
         else{
            temp[k] = arr[j];
            j++;
         }
         k++;
      }
      while (i <= mid)
      {
         temp[k] = arr[i];
         i++;
         k++;
      }
      while (j <= high)
      {
         temp[k] = arr[j];
         j++;
         k++;
      }
      for (i = low; i <= high; i++)
      {
         arr[i] = temp[i];
      }
   }
   void logic(int low, int high)
   {
```

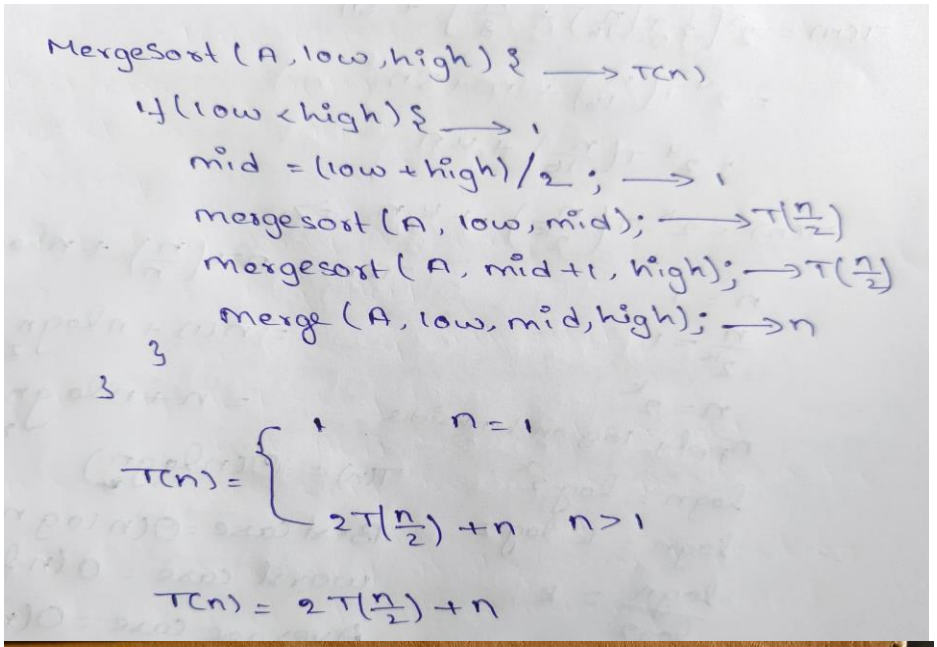
```
        if (low < high)
        {
            int mid = (low + high) / 2;
            logic(low, mid);
            logic(mid + 1, high);
            merge(low, mid, high);
        }
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main()
{
    MergeSort ms;

    ms.arrayInput();

    cout << "Before Sorting: ";
    ms.printArray();

    ms.logic(0, ms.n - 1);

    cout << "After Sorting: ";
    ms.printArray();

    return 0;
}
```

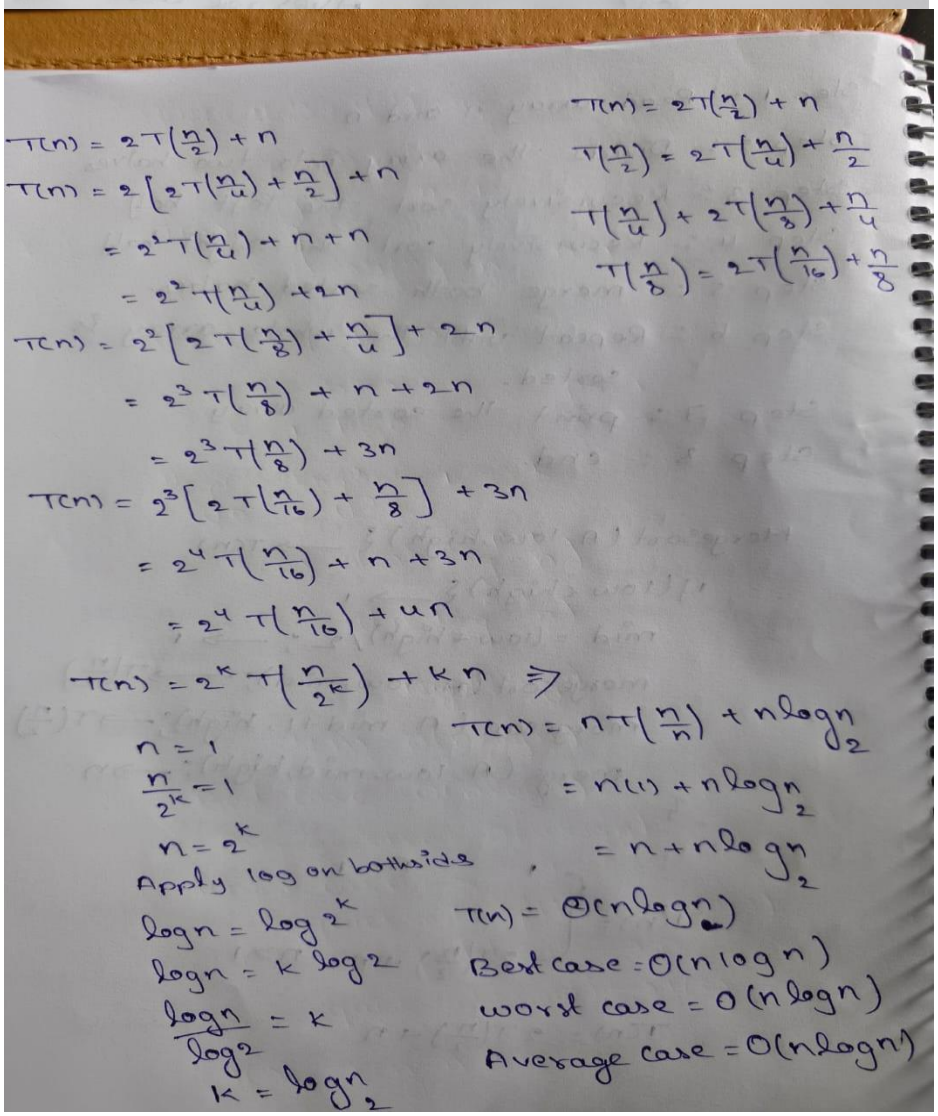
Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\mergesort.exe"
Enter number of elements: 6
Enter the elements: 15
9
6
2
22
7
Before Sorting: 15 9 6 2 22 7
After Sorting: 2 6 7 9 15 22
```

Name: Chinni Thabitha
Enrollment Number:92460118038

Time Complexity:

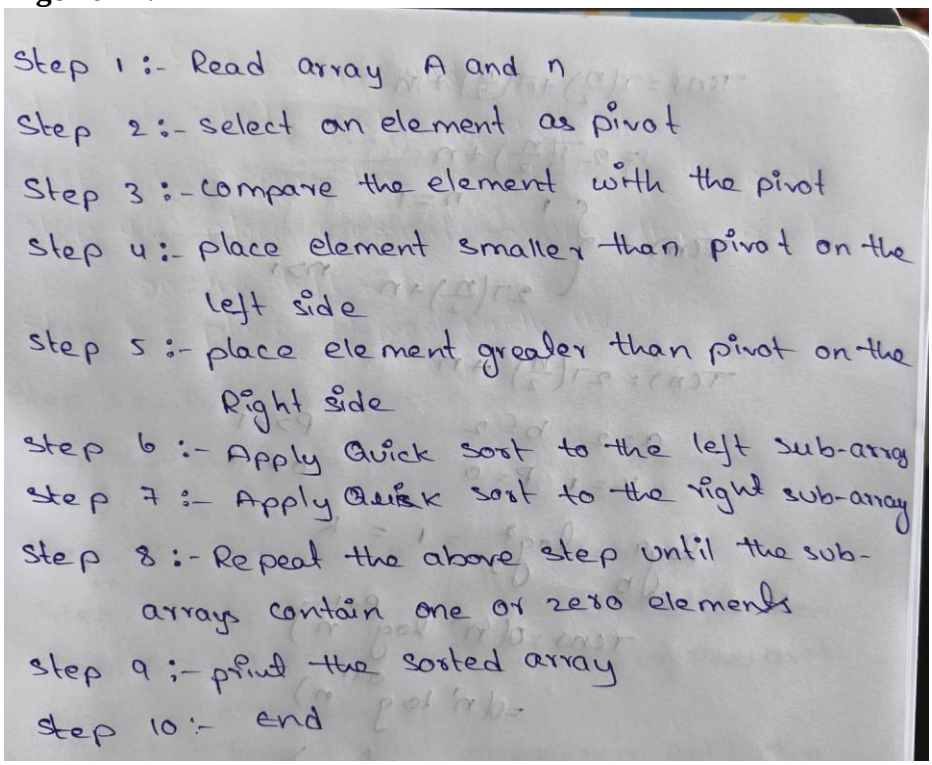




Name: Chinni Thabitha
Enrollment Number:92460118038

5)Quick Sort:

Algorithm:



Program:

```
#include <iostream>
using namespace std;
class QuickSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
```

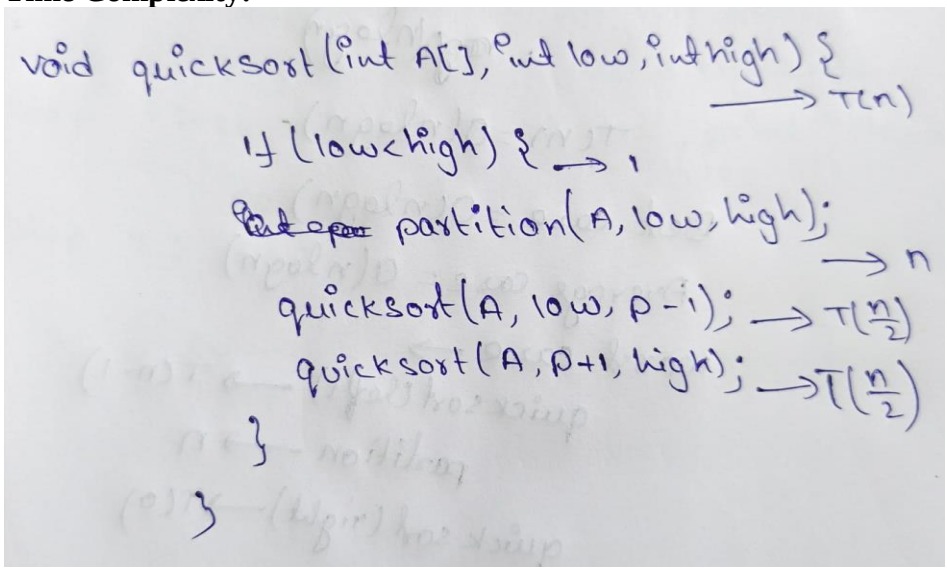
```
        cout << endl;
    }

    int partition(int arr[], int lb, int ub)
    {
        int pivot = arr[lb];
        int start = lb;
        int end = ub;

        while (start < end)
        {
            while (arr[start] <= pivot && start < ub)
            {
                start++;
            }

            while (arr[end] > pivot)
            {
                end--;
            }

            if (start < end)
            {
                swap(arr[start], arr[end]);
            }
        }

        swap(arr[lb], arr[end]);

        return end;
    }
    void quicksort(int arr[], int lb, int ub)
    {
        if (lb < ub)
        {
            int location = partition(arr, lb, ub);

            quicksort(arr, lb, location - 1);
            quicksort(arr, location + 1, ub);
        }
    }

    void sort()
    {
        quicksort(arr, 0, n - 1);
    }
};

int main()
```

```
{
    QuickSort qs;

    qs.arrayInput();

    cout << "\nBefore Sorting: ";
    qs.printArray();

    qs.sort();

    cout << "After Sorting: ";
    qs.printArray();

    return 0;
}
```

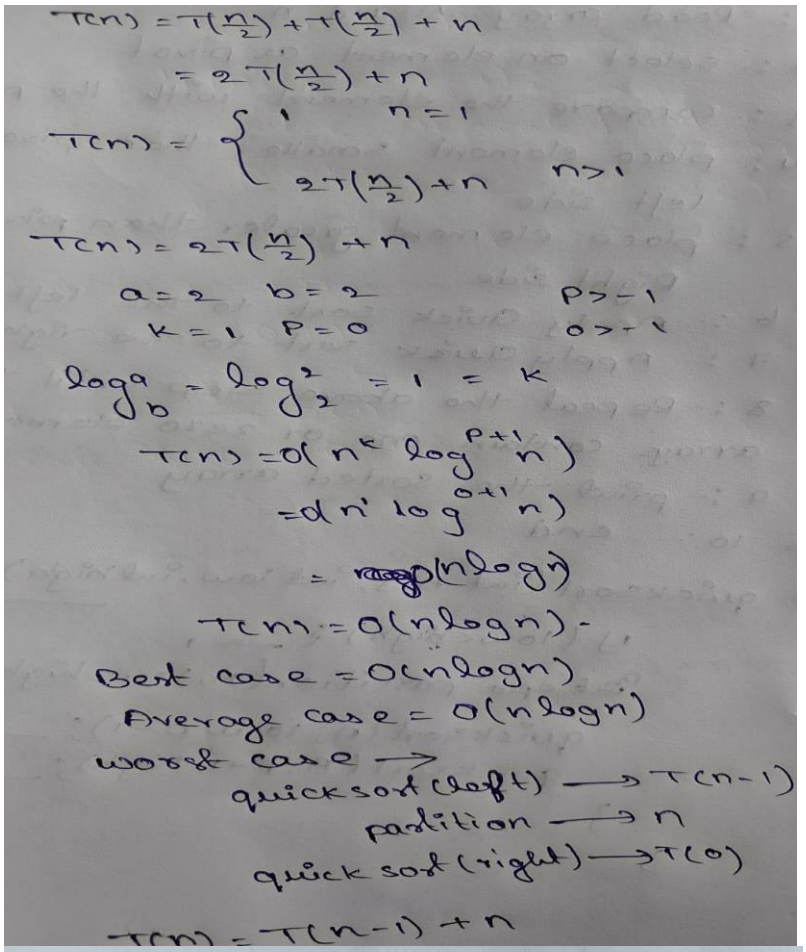
Output:

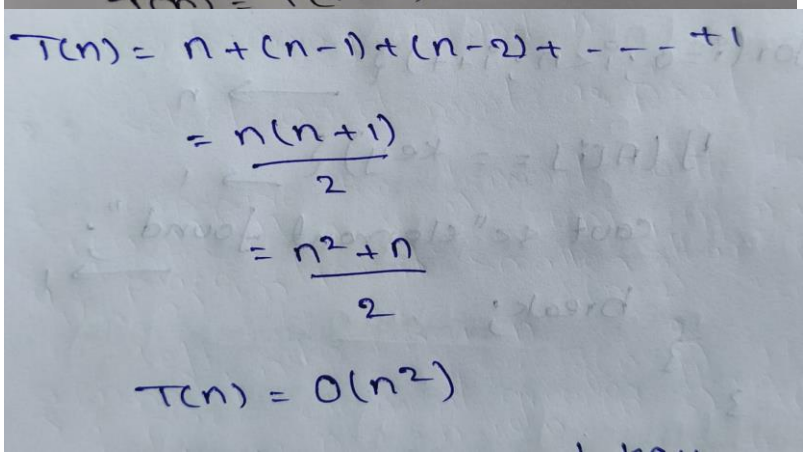
```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\quicksort.exe"
Enter number of elements: 6
Enter the elements: 15
9
3
4
23
2

Before Sorting: 15 9 3 4 23 2
After Sorting: 2 3 4 9 15 23
```

Time Complexity:



Name: Chinni Thabitha
Enrollment Number:92460118038





Conclusion:

The implementation and time analysis of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort helps us understand how different sorting algorithms perform for different input sizes. By analyzing their best, average, and worst cases, we can choose the most suitable algorithm for a particular problem. Merge Sort and Quick Sort are generally faster for large datasets, while Bubble, Selection, and Insertion Sort are simpler and useful for small datasets.

Name: Chinni Thabitha
Enrollment Number:92460118038